

Tervezési minták egy OO programozási nyelvben

Tervezési minták

A tervezési minták sokszor felmerülő problémák leírásait adják meg úgy, hogy közben megadják a megoldás magját, amit adaptálva a konkrét rendszerhez újra és újra alkalmazhatunk.

Objektum orientált programok esetében a tervezési minta leírása megadja azokat az egymással kommunikáló objektumokat, osztályokat, amelyek együttes viselkedése az adott problémára megoldás lehet.

A minták persze nem fednek le mindent, és mindenre nem adnak megoldást, csak a gyakorta előforduló problémákra adnak megoldás javaslatot. Az is előfordulhat, hogy egy-egy speciálisabb terület saját mintákat dolgoz ki, amelyek persze nem lesznek olyan széles körben ismertek

Mi az az MVC minta?

Az MVC egy ismert tervezési minta, amit főleg webalkalmazásokban és asztali programokban használnak, és célja, hogy a szoftvert három külön részre osztja:

Model: Ez a réteg felelős az app adatainak kezeléséért. Itt tároljuk a biznisz logikát, adatbázis kapcsolatokat és minden olyan funkciót, ami közvetlenül az adatokkal foglalkozik

View (Nézet): A nézet az adatok megjelenítéséért felelős felhasználók számára. A nézet a felhasználói felületet alakítja ki, legyen szó weboldalokról, desktop alkalmazásokról vagy mobil applikációkról. A cél, hogy a felhasználó számára könnyen érthető és használható felületet biztosítsunk.

Controller (Vezérlő): A vezérlő a felhasználói interakciókat kezeli, például gombnyomásokat vagy adatbevitelt, és frissíti a modellt vagy a nézetet. Itt dől el, hogy milyen válaszokat ad a rendszer a felhasználói igényekre.

A Tervezési minták elemei

1. **Név:** A tervezési minták megadásakor meg kell adjunk annak nevét. Ez fontosabb elem, mint ahogy azt gondolnánk, sokszor a név alapján már beazonosítható az, hogy a minta mely problémára is próbál megoldást nyújtani.
2. **Probléma:** Ezt mindenféleképpen tartalmaznia kell a minta leírásának, mivel tudni kell, hogy az adott minta milyen helyzetben, milyen kontextusban használható.

3. **Megoldás:** A minták általános keretet adnak a probléma megoldásához, amely az adott alkalmazás igényeihez igazítható.

Mi nem tervezési minta?

A program alapvető építőelemeit, mint az algoritmusokat, absztrakt adattípusokat nem tekintjük tervezési mintának. Míg az algoritmusok konkrét lépések sorozatát adják meg egy adott probléma megoldására, a tervezési minták általános keretet vagy megközelítést biztosítanak gyakori szoftvertervezési problémákhoz.

Példa a különbségre:

- **Algoritmus:** A *kiválasztásos rendezés* egy konkrét algoritmus, amely lépésről lépésre meghatározza, hogyan rendezzünk sorba egy tömböt.
- **Tervezési minta:** A *Strategy* tervezési minta lehetőséget ad arra, hogy különböző rendezési algoritmusokat használjunk ugyanazon interfészen keresztül, például lehetővé téve, hogy a futási időben válasszunk a *kiválasztásos rendezés* és a *gyorsrendezés* között.

Minták katalógusa

Általában két szempont alapján csoportosítjuk:

1. **Rendeltetés szerint:** Rendeltetés szerint lehetnek gyártási mintáink, amelyek alapvetően az objektumoknak a létrehozási módjáért felelősek, lehetnek szerkezetik, amik meghatározzák objektumok vagy osztályok felépítését, illetve lehetnek viselkedési minták, amik osztályok és objektumok közötti kölcsönhatásokat írnak le, illetve azt, hogyan osztják meg egymás között a felelősségeket.
2. **Hatáskör szerint:** Hatáskör szerint csoportosítva a mintákat megkülönböztetünk olyanokat, amik osztályok közötti statikus, vagy amik objektumok között levő dinamikus kapcsolatokat írnak le.

A gyártási minták

1. **Abstract Factory** (Kit) - Elvont gyár: Kapcsolódó vagy egymástól függő objektumok családjának létrehozására szolgáló interfészt biztosít a konkrét osztályok megadása nélkül.

Használat:

- Platformfüggetlen UI elemek létrehozása.
- Olyan rendszerek, ahol több termékcsaládot kell kezelni, például különböző adatbázis-illesztők (MySQL, PostgreSQL)

Példa: Egy grafikus felületre vonatkozó absztrakt gyár, amely létrehozza a gombokat és menüket az adott operációs rendszer alapján.

2. **Builder** - Építő: Összetett objektum létrehozását biztosítja az elemeinek létrehozásával és összerakásával. Az építési folyamat szétválik a végterméktől, így ugyanaz az építési folyamat különböző megvalósításokat eredményezhet.

Használat:

- Konfigurálható termékek, pl. autók összeállítása vagy számítógépek összeállítása.
- Dokumentumok generálása többféle formátumban (PDF, HTML).

Példa: Egy autó-összeállító program, ahol az autók különböző konfigurációkkal készülhetnek.

3. **Prototype** - Prototípus: Prototípus példány használatával egy alap objektumot, az újabb objektumok létrehozását pedig ennek a prototípusnak a lemásolásával állítja elő.

Használat:

- Objektumok klónozása nagy számú példány esetén.
- Olyan rendszerek ahol a konfigurációkat sablonként használják.

Példa: Egy játékban a különböző karakterek alap prototípusként vannak definiálva, és ezeket klónozzák, majd testreszabják.

4. **Singleton** - Egyke: Egy osztályból csak egy példányt engedélyez, és ehhez globális hozzáférést ad.

Használat:

- Konfigurációk, amelyek az egész alkalmazásban egységesek.
- Adatbázis-kapcsolatok megosztása.

Példa: Egy naplózó osztály, amely biztosítja, hogy minden komponens ugyanazt a naplóállományt használja.

5. **Factory Method** (Virtual Constructor) - Gyártófüggvény: Interfészt definiál egy objektum létrehozásához, de az alosztályokra bízta, melyik osztályt példányosítják. A

factory method megengedi az osztályoknak, hogy a példányosítást az alosztályokra ruházzák át.

Használat:

- Amikor a létrehozandó objektum típusa a futásidőben dől el.
- Amikor az osztály hierarchiájában változó objektumokat kell létrehozni

Példa: Egy hitelkezelő rendszer, ahol különböző típusú hitelszámlák (pl. személyi kölcsön, jelzáloghitel) dinamikusan jönnek létre a megfelelő paraméterek alapján.

A szerkezeti minták

1. **Adapter** (Wrapper) - Illesztő: Az adott osztály interfészét átalakítja, hogy az kompatibilissé (használhatóvá) váljon más rendszerek részére.

Használat:

- Régi kód új környezetbe való illesztése.
- Eltérő adatbázisok vagy API-k illesztése

2. **Bridge** (Handle, Body) - Híd: Az elvont ábrázolás (absztrakció) és implementáció közötti csatolást lazítja, hogy a kettő egymástól függetlenül legyen változtatható.

Használat:

- GUI elemek, ahol az absztrakció (pl. Button) és az implementáció (pl. Windows vagy Mac rajzolósi mód) függetlenül változtatható.

3. **Composite** - Összetétel: Rész-egész szerkezetek leképezését valósítja meg az objektum-hierarchiára a rész és egész azonos kezelésével.

Használat:

- Fájlrendszerek, UI hierarchia, menük kezelése.

4. **Decorator** (Wrapper) - Díszítő: További felelősségeket/tulajdonságokat csatol dinamikusan az objektumhoz.

Használat:

- Fájlműveletek kibővítése naplózással.
- UI komponensek további stílusjegyekkel való bővítése.

5. **Facade** - Homlokzat: Egy alrendszerben különböző interfészek egy halmazához egységes interfészt biztosít. A módszerrel magasabb szintű interfészt határozzunk meg, amelynek révén az adott alrendszer könnyebben használhatóvá válik.

Használat:

- Komplex API-k egyszerűsítése, mint például adatbázis-műveletek vagy grafikus motorok.

6. **Flyweight** - Pehelysúlyú: Nagy számú apró, megosztott objektum kezelését teszi hatékonyá. Akkor használatos, ha nagy (nagyságrendileg száz, vagy ezer) számú objektumunk van, amelyek nagyon hasonlóak. Ilyenkor ez a minta csökkentheti a memória felhasználást.

Használat:

- Karakterek megjelenítése szövegszerkesztőben.
- Játékokban több ezer azonos textúrájú objektum (pl. fák) kezelése.

7. **Proxy** (Surrogate) - Helyettes: Egy adott objektumot egy helyettesítő objektummal vált fel, amely szabályozza az eredeti objektumhoz történő hozzáférést is. Gyakorlatilag a proxyval egy köztes réteg kerül a kliens és az objektum közé. Így az eredeti objektumhoz való hozzáférést kontrollálhatjuk.

Használat:

- Távoli objektumok kezelése (Remote Proxy).
- Hozzáférés ellenőrzése (Protection Proxy).

Viselkedési minták

1. **Interpreter** - Értelmező: Egy adott nyelv nyelvtanát reprezentálja megadva azt az értelmezőt is, amely értelmezni tudja az adott nyelven leírt mondatokat.

Használat:

- Matematikai kifejezések értelmezése
- SQL-parancsok feldolgozása, konfigurációs fájlok értelmezése.

2. **Template Method** - Sablonfüggvény: Egy algoritmus vázát definiálja, amelyben bizonyos műveleteket csak absztrakt módon definiál, megvalósításukért csak a származtatott osztályok lesznek felelősek.

Használat:

- Játékok szabályrendszerének implementálása.
- Adatfeldolgozási folyamatok

3. **Chain of Responsibility** - Felelősséglánc: A kérés/parancs küldője és végrehajtója közötti közvetlen csatolást megszünteti úgy, hogy a kérést feltételezhetően kezelni tudó objektumokat láncba állítja, és a kérés addig halad a kiszolgálók láncolatán, amíg azt valamelyik le nem kezeli.

Használat:

- Hibaelhárítás.
- Űrlap-adatok validációja.

4. **Command** (Action, Transaction) - Parancs: Kéréseket objektumként ábrázolja azok sorba állíthatósága, naplózhatósága végett.

Használat:

- Visszavonható műveletek.
- Gombokhoz kötött műveletek

5. **Iterator** (Cursor) - Iterator: Tároló-objektum elemeinek sorozatos elérését teszi lehetővé a reprezentációtól függetlenül.

Használat:

- Egy lista vagy gyűjtemény bejárása, például adatbázis rekordok feldolgozása.

6. **Mediator** - Közvetítő: Objektumok együttműködési módját változtatja meg egy egységbezáró objektum közbeékelésével, amely által a két objektum közötti eredeti csatolás megszűnik.

Használat:

- Chat alkalmazás felhasználói közötti kommunikáció. □ GUI elemek eseményeinek központosított kezelése.

7. **Memento** (Token) - Emlékeztető: Az egységbe zárás megsértése nélkül kinyeri és elmenti egy objektum belső állapotát, hogy az később ebbe az állapotba visszaállítható legyen.

Használat:

- Szövegszerkesztő visszavonási funkciója.
- Játékállások kezelése.

8. **Observer** (Dependents, Publish-Subscribe) - Megfigyelő: Egy objektum állapotát megfigyeli, az objektum állapotának megváltozásáról pedig a tőle függő objektumokat értesíti.

Használat:

- Hírlevél-szolgáltatás értesítései.
- GUI események kezelése

9. **State** (Objects for States) - Állapot: Adott objektum számára engedélyezi, hogy belső állapotának megváltozásával megváltoztathassa viselkedését is. Az objektum ekkor látszólag módosítja az osztályát.

Használat:

- Baki számla állapotkezelése.
- Játék karakterek különböző módjai.

10. **Strategy** (Policy) - Stratégia: Meghatároz egy algoritmus családot, amelyben a több, azonos feladatú, de különböző algoritmus egységbezárásával azok felcserélhetővé válnak.

Használat:

- Rendezési algoritmusok választása futásidőben.

- Adatfeldolgozási stratégiák.

11. **Visitor** - Látogató: Egy adott objektum-hierarchián végezhető műveletsort reprezentál, a hierarchia bejárásával ezen műveletek elvégezhetőekké válnak a hierarchia elemein anélkül, hogy a hierarchiát alkotó osztályokat megváltoztatnánk.

Használat:

- Compiler AST (Abstract Syntax Tree) bejárása.
- Fájlstruktúrák különböző műveleteinek végrehajtása.

Készítette: Sztányi György (ACBQWU)

PTI-BSC